

## 1. Maximum sum subarray of size k.

**Question:** Given an array of integers (arr) and a window size k, find the subarray (a continuous block of elements) of size k that has the largest possible sum.

**Simple Explanation:** We use the **Sliding Window** technique. We first calculate the sum of the very first window (size k). Then, we move the window one step to the right by **subtracting the element that slides out** on the left and **adding the new element that slides in** on the right. We continuously update the maximum sum found across all windows.

```
public class MaxSumSubarray {  
    /**  
     * @param arr The input array of integers.  
     * @param k The size of the subarray window.  
     * @return The maximum sum found for any subarray of size k.  
     */  
    public static int findMaxSumSubarray(int[] arr, int k) {  
        if (arr == null || k > arr.length) {  
            throw new IllegalArgumentException("Invalid input: Array is  
null or k is too large.");  
        }  
  
        int currentSum = 0;  
        int maxSum = 0;  
  
        // 1. Calculate sum of the first window (initial setup)  
        for (int i = 0; i < k; i++) {  
            currentSum += arr[i];  
        }  
        maxSum = currentSum;  
  
        // 2. Slide the window across the rest of the array  
        for (int i = k; i < arr.length; i++) {  
            // Subtract the element that slides out (arr[i - k])  
            // Add the new element that slides in (arr[i])  
            currentSum = currentSum - arr[i - k] + arr[i];  
  
            // Update the global maximum sum  
            maxSum = Math.max(maxSum, currentSum);  
        }  
  
        return maxSum;  
    }  
}
```

## 2. Longest substring without repeating characters.

**Question:** Given a string *s*, find the length of the longest substring (a continuous block of characters) that does not contain any repeating characters.

**Simple Explanation:** We use the **Sliding Window** technique with a **HashSet** to track unique characters within the current window. We expand the window (right pointer) as long as we find unique characters. If we find a repeat, we shrink the window (left pointer) by removing characters from the left until the duplicate is gone. We update the maximum length found at every step.

```
import java.util.HashSet;
import java.util.Set;

public class LongestUniqueSubstring {
    /**
     * @param s The input string.
     * @return The length of the longest substring without repeating
     * characters.
     */
    public static int lengthOfLongestSubstring(String s) {
        if (s == null || s.isEmpty()) {
            return 0;
        }

        // Set to store characters in the current window for quick lookup
        Set<Character> charSet = new HashSet<>();
        int left = 0; // Left pointer of the window
        int maxLength = 0; // Stores the result

        // Right pointer expands the window
        for (int right = 0; right < s.length(); right++) {
            char currentChar = s.charAt(right);

            // If the character is already in the set, shrink the window
            // from the left
            while (charSet.contains(currentChar)) {
                charSet.remove(s.charAt(left));
                left++;
            }

            // Add the new unique character to the set
            charSet.add(currentChar);

            // Update the maximum length found so far
            maxLength = Math.max(maxLength, right - left + 1);
        }

        return maxLength;
    }
}
```